

torch.autograd.functional.jacobian#

<https://pytorch.org/docs/stable/generated/torch.autograd.functional.jacobian>

Description:

Compute the Jacobian of a given function. `func` (function) – a Python function that takes Tensor inputs and returns a tuple of Tensors or a Tensor. `inputs` (tuple of Tensors or Tensor) – inputs to the function `func`. `create_graph` (bool, optional) – If True, the Jacobian will be computed in a differentiable manner. Note that when `strict` is False, the result can not require gradients or be disconnected from the inputs. Defaults to False. `strict` (bool, optional) – If True, an error will be raised when we detect that there exists an input such that all the outputs are independent of it. If False, we return a Tensor of zeros as the jacobian for said inputs, which is the expected mathematical value. Defaults to False.

Function Signature

```
torch.autograd.functional.jacobian(func, inputs,  
create_graph=False, strict=False, vectorize=False,  
strategy='reverse-mode')[source]#
```

Parameters

Returns

Return type

Returns:

Jacobian (Tensor or nested tuple of Tensors)

Code Examples

```
>>> def exp_reducer(x):  
...     return x.exp().sum(dim=1)  
>>> inputs = torch.rand(2, 2)  
>>> jacobian(exp_reducer, inputs)  
tensor([[[1.4917, 2.4352],  
         [0.0000, 0.0000]],  
        [[0.0000, 0.0000],  
         [2.4369, 2.3799]]])
```

```
>>> jacobian(exp_reducer, inputs, create_graph=True)
tensor([[[1.4917, 2.4352],
          [0.0000, 0.0000]],
        [[0.0000, 0.0000],
          [2.4369, 2.3799]]], grad_fn=<ViewBackward>)
```

```
>>> def exp_adder(x, y):
...     return 2 * x.exp() + 3 * y
>>> inputs = (torch.rand(2), torch.rand(2))
>>> jacobian(exp_adder, inputs)
(tensor([[2.8052, 0.0000],
          [0.0000, 3.3963]]),
 tensor([[3., 0.],
          [0., 3.])))
```

```
>>> def linear_model(x):
...     W = torch.tensor([[2.0, -1.0], [0.0, 1.0]])
...     b = torch.tensor([1.0, 0.5])
...     return x @ W.T + b
```

```
>>> x = torch.randn(4, 2, requires_grad=True)
>>> jac = jacobian(linear_model, x, vectorize=True)
>>> jac.shape
torch.Size([4, 2, 4, 2])
```

Detailed Documentation

Documentation

Compute the Jacobian of a given function.

`func` (function) – a Python function that takes Tensor inputs and returns a tuple of Tensors or a Tensor.

`inputs` (tuple of Tensors or Tensor) – inputs to the function `func`.

`create_graph` (bool, optional) – If True, the Jacobian will be computed in a differentiable manner. Note that when `strict` is False, the result can not require gradients or be disconnected from the inputs. Defaults to False.

`strict` (bool, optional) – If True, an error will be raised when we detect that there exists an input such that all the outputs are independent of it. If False, we return a Tensor of zeros as the jacobian for said inputs, which is the expected mathematical value. Defaults to False.

`vectorize` (bool, optional) – This feature is experimental. Please consider using `torch.func.jacrev()` or `torch.func.jacfwd()` instead if you are looking for something less experimental and more performant. When computing the jacobian, usually we invoke `autograd.grad` once per row of the jacobian. If this flag is True, we perform only a single `autograd.grad` call with `batched_grad=True` which uses the `vmap` prototype feature. Though this should lead to performance improvements in many cases, because this feature is still experimental, there may be performance cliffs. See `torch.autograd.grad()`'s `batched_grad` parameter for more information.

`strategy` (str, optional) – Set to "forward-mode" or "reverse-mode" to determine whether the Jacobian will be computed with forward or reverse mode AD. Currently, "forward-mode" requires `vectorized=True`. Defaults to "reverse-mode". If `func` has more outputs than inputs, "forward-mode" tends to be more performant. Otherwise, prefer to

use "reverse-mode".

if there is a single input and output, this will be a single Tensor containing the Jacobian for the linearized inputs and output. If one of the two is a tuple, then the Jacobian will be a tuple of Tensors. If both of them are tuples, then the Jacobian will be a tuple of tuple of Tensors where `Jacobian[i][j]` will contain the Jacobian of the *i*th output and *j*th input and will have as size the concatenation of the sizes of the corresponding output and the corresponding input and will have same dtype and device as the corresponding input. If strategy is forward-mode, the dtype will be that of the output; otherwise, the input.

Jacobian (Tensor or nested tuple of Tensors)